B.Tech (ECE/BML)
Fall Sem 2025-26
Course Project

BECE312L – Robotics & Automation
C2 Slot - VL2025260102895

Title of the Project

ESP-32 based Wireless Car Control with Fire Detection

Team Members Details

Stavyansh Mishra (24BEC0169)

Aryan Singh (24BEC0201)

Saatwik Gupta (24BEC0590)

Faculty

Dr.Vijendra Babu D
Associate Professor Grade 1
Dept. of Embedded Technology

School of Electronics Engineering, Vellore Institute of Technology.

Outline

- Introduction
- Objectives
- Literature Survey
- System Architecture
- Hardware Design
- Software Design
- Implementation
- Testing & Results
- Applications
- Challenges & Limitations
- Scope for Improvement
- Sustainable Development Goals
- Timeline(Week wise)
- Acknowledgments
- References

Introduction

- This project presents the design and implementation of a **wirelessly controlled robotic car** that moves in response to **WASD key commands** using **ESP-NOW communication** between two ESP32 boards.

- The system is **unidirectional**, meaning commands are only sent from the **controller (transmitter)** to the **car (receiver)**.
  The receiver drives motors via an **L298N motor driver**, and uses a **flame sensor** to detect fire and trigger a **buzzer** as an alert.

- Initially, the car was tested in a **wired setup using Arduino UNO** to verify basic motor control and flame detection before moving to a fully wireless ESP32 configuration.

Objectives

- To design a robotic car that can be controlled wirelessly using the ESP-NOW protocol.
- To allow motion control using WASD Key input
- To detect nearby fire using flame sensor and alert using a buzzer
- To demonstrate a low-cost, scalable, and safe automation prototype

Literature Review

- Conventional wireless robotic systems use Wi-Fi, Bluetooth, or RF modules such as HC-05 or Nrf modules
- These methods, while reliable, often have pairing delays, higher latency, or limited range.
- **ESP-NOW**, developed by Espressif, provides a **fast, connectionless, unidirectional communication protocol** ideal for real-time robotics control.
- Prior projects have explored wireless data transfer and remote control, but integrating **motion control**, **flame detection**, and **alerting** in a simple ESP32-based system is relatively new.
  This project builds upon these concepts and simplifies the implementation for educational and safety use cases.

System Architecture

- The system consists of two major modules:
- **1. Transmitter (Controller)**
- ESP32 sends specific characters (‘W’, ‘A’, ‘S’, ‘D’, ‘Space’) via ESP-NOW.
- Commands are transmitted one-way to the receiver.
- **2. Receiver (Robot Car)**
- Receives commands and controls two DC motors using L298N motor driver.
- Continuously monitors the flame sensor.
- Activates the buzzer when fire is detected.
- No acknowledgment or return communication is implemented.

Hardware Design

- **Components Used:**
- 2 × ESP32 Development Boards (Sender & Receiver)
- 1 × L298N Motor Driver Module
- 2 × DC Motors + Car Chassis
- 1 × Flame Sensor
- 1 × Active Buzzer
- 1 × Power Supply (Battery Pack 7.4–12V)
- Jumper Wires, Breadboard
- **Wiring Overview:**
- ESP32 → L298N (IN1–IN4 for direction, ENA/ENB for speed control)
- Flame Sensor → Analog pin of ESP32
- Buzzer → Digital output pin
- Common ground across ESP32, motor driver, and power source

Software Design

- **Programming Environment:** Arduino IDE
  Language: Embedded C / Arduino C++
- **Transmitter Code:**

Reads key input (‘W’, ‘A’, ‘S’, ‘D’, ‘Space’) from Serial Monitor or keyboard emulator.

Sends the corresponding character via ESP-NOW to the receiver ESP32.

- **Receiver Code:**

Receives the command character and drives motors accordingly.

Continuously checks the flame sensor value.

Triggers the buzzer if fire intensity crosses the threshold.

Sender Code Snippet

sender.ino

```
#include <esp_now.h>
#include <WiFi.h>
uint8_t receiverAddress[] = {0x08, 0xA6, 0xF7, 0xB1, 0x12, 0xF8};
typedef struct struct_message {
  char command;
} struct_message;
struct_message myData;
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
  Serial.print("Send Status: ");
  if (status == ESP_NOW_SEND_SUCCESS) {
    Serial.println(" Success");
  } else {
    Serial.println(" Failed");
  }
}
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n================================");
  Serial.println("ESP32 Wireless Car Controller");
  Serial.println("================================");
  WiFi.mode(WIFI_STA);
  Serial.print("Sender MAC: ");
  Serial.println(WiFi.macAddress());
  Serial.println();
  if (esp_now_init() != ESP_OK) {
    Serial.println(" ERROR: ESP-NOW Init Failed!");
    return;
  }
  Serial.println(" ESP-NOW Initialized");
  esp_now_register_send_cb(OnDataSent);
  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, receiverAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;
```

sender.ino

```
  peerInfo.encrypt = false;
  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println(" ERROR: Failed to add peer!");
    return;
  }
  Serial.println(" Peer Added");
  Serial.println("\n================================");
  Serial.println("Controller Ready!");
  Serial.println("================================");
  Serial.println("Commands:");
  Serial.println("  W - Move Forward");
  Serial.println("  A - Turn Left");
  Serial.println("  S - Move Backward");
  Serial.println("  D - Turn Right");
  Serial.println("  X - Stop");
  Serial.println("================================\n");
}
void loop() {
  if (Serial.available() > 0) {
    char input = Serial.read();
    if (input >= 'a' && input <= 'z') {
      input = input - 32;
    }
    if (input == 'W' || input == 'A' || input == 'S' || input == 'D' || input == 'X') {
      sendCommand(input);
    }
  }
}
void sendCommand(char cmd) {
  Serial.print("Sending: ");
  switch (cmd) {
    case 'W': Serial.println("FORWARD "); break;
    case 'A': Serial.println("LEFT "); break;
    case 'S': Serial.println("BACKWARD "); break;
    case 'D': Serial.println("RIGHT "); break;
    case 'X': Serial.println("STOP "); break;
  }
  myData.command = cmd;
  esp_err_t result = esp_now_send(receiverAddress, (uint8_t *)&myData, sizeof(myData));
  if (result != ESP_OK) {
    Serial.println(" ERROR: Sending failed!");
  }
  delay(50);
}
```

Receiver Code Snippet

reviever.ino

```
#include <esp_now.h>
#include <WiFi.h>
#define IN1 13   // Left motor input 1
#define IN2 12   // Left motor input 2
#define IN3 14   // Right motor input 1
#define IN4 27   // Right motor input 2
#define FLAME_SENSOR 34   // IR flame sensor output pin
#define BUZZER_PIN   26   // Active buzzer pin
char command = ' ';
bool flameDetected = false;
void stopCar() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}
void moveCar(char cmd) {
  cmd = tolower(cmd);
  switch (cmd) {
    case 'w': // Forward
      digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
      digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
      break;
    case 's': // Backward
      digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
      digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
      break;
    case 'a': // Left
      digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
      digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
      break;
    case 'd': // Right
      digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
      digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
      break;
    default: // Stop
      stopCar();
      break;
  }
}
```

reviever.ino

```
      break;
    default: // Stop
      stopCar();
      break;
  }
}
void onDataRecv(const esp_now_recv_info * info, const uint8_t *incomingData, int len) {
  command = (char)*incomingData;
  Serial.print("Received: ");
  Serial.println(command);
  moveCar(command);
}
void setup() {
  Serial.begin(115200);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  stopCar();
  pinMode(FLAME_SENSOR, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  WiFi.mode(WIFI_STA);
  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed!");
    return;
  }
  esp_now_register_recv_cb(onDataRecv);
  Serial.println("Receiver ready on new pins + flame sensor.");
}
void loop() {
  int flameValue = digitalRead(FLAME_SENSOR);
  if (flameValue == LOW) {
    if (!flameDetected) {
      flameDetected = true;
      Serial.println(" Flame detected! Buzzer ON");
    }
    digitalWrite(BUZZER_PIN, HIGH);
  } else {
    if (flameDetected) {
      flameDetected = false;
      Serial.println(" No flame. Buzzer OFF");
    }
    digitalWrite(BUZZER_PIN, LOW);
  }
  delay(100);
}
```

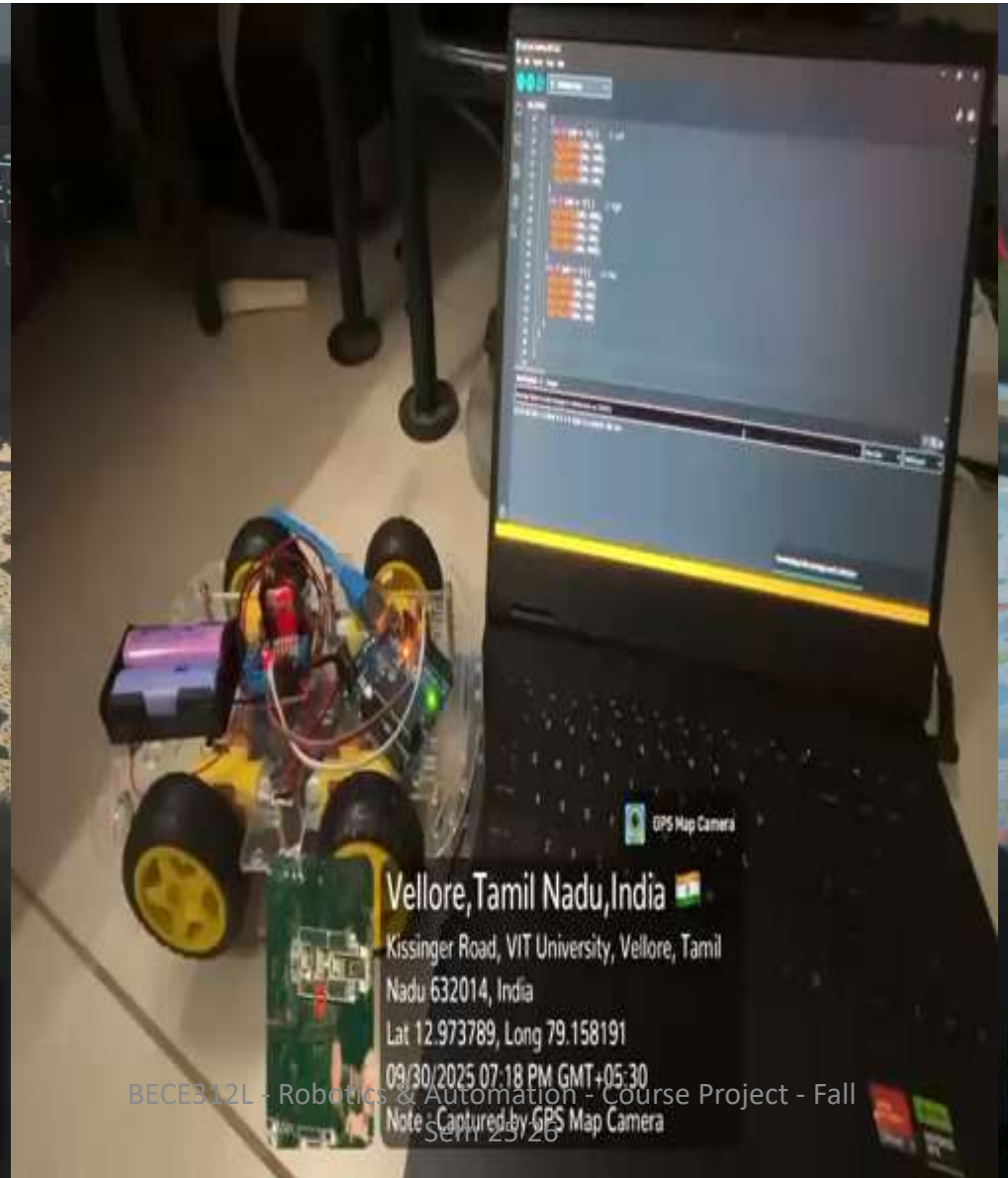
Implementation

- **Phase 1:** Developed a wired prototype using **Arduino UNO** and **L298N** to test motor control.
- **Phase 2:** Configured two ESP32 boards for **ESP-NOW one-way communication**.
- **Phase 3:** Tested the working of flame sensor and implemented it with a active buzzer

Testing & Results

Wireless Range: about 30 meters stable in open area.
Response Time: <100 ms between keypress and movement.
Flame Detection: Accurate trigger within 1 second of flame presence.
Result: The robot responded reliably to all WASD inputs and effectively detected fire, sounding the buzzer immediately.
The overall performance was consistent after minor calibration of motor control and power management

Video of the Project



Applications

- Fire detection and response robot prototype
- Basic framework for home or industrial safety robots
- Foundational system for smart surveillance and rescue robots
- Educational robotics and IoT experimentation platform

Challenges & Applications

- Technical **Component Failures:** During testing, **one ESP32 module and two L298N motor drivers were burnt** due to power surges and reverse polarity connections.
- **Wiring complexity:** Loose jumper wires caused intermittent signal loss.
- **Power Issues:** Battery voltage fluctuations affected motor torque.
- **Sensor limitations:** Flame sensor's detection range is limited (~20–30 cm) and affected by sunlight.
- **Communication:** ESP-NOW works only in one direction; no acknowledgment or feedback possible.
- **Range Limitation:** Stable range limited to around 50 meters.
- **Initial Debugging:** Motor driver heating and current overload led to frequent testing delays.

Future Scope

- Upgrade to **bidirectional ESP-NOW** or **LoRa communication** for feedback-based control.
- Add **ultrasonic sensors** for obstacle avoidance.
- Include a **camera module (ESP32-CAM)** for visual fire tracking.
- Use **TB6612FNG motor driver** for higher efficiency and less heating.
- Integrate **solar power** for longer operation.
- Create a **mobile app interface** for control instead of serial input.

Sustainable Development Goals

- **SDG 9:** Encourages innovation in affordable robotics and embedded systems.
- **SDG 11:** Promotes safer and more sustainable communities through fire detection automation.
- **SDG 13:** Contributes to climate action by enabling early fire detection and response.
- **SDG 12:** Supports responsible energy usage through efficient IoT operation.

Conclusion

- The project successfully demonstrates a **wirelessly controlled car** using **ESP-NOW** for unidirectional communication between two ESP32 boards.
  It efficiently responds to **WASD commands**, detects fire through a **flame sensor**, and alerts via a **buzzer**.
- Starting with a **wired Arduino UNO prototype**, the system was later upgraded to a fully wireless model
- Despite setbacks like **burnt components**, the final prototype achieved **stable operation, quick response, and accurate fire detection**, proving its potential for **safety and educational applications**.

Timeline

Week	Task
1	Component procurement and initial circuit testing with Arduino UNO
2	Motor driver testing and basic movement validation
3	Setting up unidirectional ESP-NOW communication
4	Integrating flame sensor and buzzer system
5	Field testing, tuning, and bug fixes
6	Replacing burnt ESP32 and L298N modules
7	Documentation and final presentation preparation

References

1. Espressif Systems – *ESP-NOW Protocol Documentation*.
2. Arduino Documentation – *ESP32 Wi-Fi and PWM Library Reference*.
3. IEEE Xplore – *Wireless Robotic Fire Detection Systems (2023)*.
4. TutorialsPoint – *L298N Motor Driver with ESP32*.
5. ResearchGate – *Flame Sensor Interfacing and IoT-Based Safety Systems*.

Acknowledgments

- We would like to express our heartfelt gratitude to our faculty mentor **Dr. Vijendra Babu D**, for his continuous guidance and support. We also thank our college for giving us the knowledge necessary and opportunity to apply our learning in real world projects .

THANK YOU